SENG402 – GRADUATION PROJECT

Software Requirements Specification Document

By Sinem Coşkun, Al-baraa Abdullah Abdullah Abdullah Al-yadumi

TABLE OF CONTENTS

1. Introduction 4
1.1 Purpose 4
1.2 Scope 4
1.3 Definitions, Acronyms, and Abbreviations 5
1.4 Document Overview 6
2. Overall Description 7
2.1 Product Perspective 7
2.2 Product Functions 8
2.3 User Classes and Characteristics 8
2.4 Operating Environment 8
2.5 Design and Implementation Constraints 9
2.6 Assumptions and Dependencies 9
3. System Features (Functional Requirements) 10
3.1 Edge Honeypot Operation 10
3.2 Log Collection and Aggregation 10
3.3 MQTT-Based Log Transmission 11
3.4 Central Log Storage (SQLite) 12
3.5 Offline Session Construction 12
3.6 Feature Extraction and Rule-Based Labeling 13
3.7 Data Leakage Prevention 13
3.8 Machine-Learning-Based Attack Classification 14
3.9 LLM-Powered Cowrie Deception (Integration Layer) 15
4. External Interface Requirements 15
4.1 User Interfaces 15
4.2 Hardware Interfaces 16
4.3 Software Interfaces 17
4.4 Communication Interfaces 17
5. Non-Functional Requirements 18
5.1 Performance Requirements 18

5.2 Reliability Requirements 19
5.3 Security Requirements 19
5.4 Usability Requirements 19
5.5 Maintainability Requirements 19
5.6 Extensibility Requirements 20
5.7 Portability Requirements 20
6. Other Requirements 20
6.1 Ethical and Privacy Considerations 20
6.2 Monitoring and Logging Requirements 21
6.3 Data Handling and Output Requirements 21
7. Appendices 22
7.1 Appendix A — Glossary 22
7.2 Appendix B — Feature List 23
Dependent Variables (Classification Targets) 24
7.3 Appendix C — Project Schedule Overview 24
7.4 Appendix D — System Architecture Diagram 25

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to describe the software requirements of the project titled:

"Detecting Protocol Tunneling Activities and Cyber Attacks with Machine Learning-Based IoT Honeypot."

The project aims to develop a comprehensive behavioral intelligence and deception platform that runs on an IoT edge device (Raspberry Pi) and a central analysis machine. The system:

- Collects real attack traffic through honeypot systems (Cowrie, Honeytrap, Suricata) and stores it in a central SQLite database.
- Performs offline, session-level behavioral analysis on the collected logs to detect tunneling activities and classify cyber attacks across 11 multi-label categories (including lateral movement, data exfiltration, reconnaissance, etc.) and bot vs. human behavior.
- Implements a strict Data Leakage Check to ensure machine learning models learn from genuine behavioral features (SAFE features) rather than deterministic shortcuts.
- Evaluates all machine learning classifiers using the Macro-F1 score (aiming for >80-90%) instead of Accuracy, due to severe class imbalance.
- Enhances the Cowrie honeypot with an LLM-Powered Deception Pipeline (using fine-tuned Phi-3) to dynamically generate realistic Linux terminal responses for unrecognized commands, thereby increasing attacker engagement and delaying honeypot fingerprinting.

This SRS defines what the system shall do, which data it shall process, which outputs it shall produce and which constraints it shall operate under, so that the implementation and evaluation can follow a clear specification.

1.2 Scope

The proposed system is an IoT honeypot–based monitoring and analysis platform built around a Raspberry Pi 4 Model B device.

Within the scope of the project:

- A Raspberry Pi is configured as an edge device exposed to the Internet through a guest network and a DMZ structure, without being connected to a production network.
- On this device, three open-source monitoring systems are installed:
    - Cowrie for SSH attack logging,
    - Honeytrap for HTTP and FTP service emulation,
    - Suricata as an IDS analyzing general network traffic.

- These systems generate log files that are transmitted to a central analysis computer using the MQTT protocol, where the logs are stored in a SQLite database.
- In the central environment, the data analysis is executed as an offline pipeline (Session Intelligence & ML Preparation). Raw events are grouped into temporally coherent sessions (e.g., 30-minute inactivity threshold) and saved as Parquet/JSON formats.
- During feature extraction, 52 raw features are generated. A rigorous Data Leakage Fix separates these into 24 SAFE behavioral features strictly used for ML training, and 28 heuristic/leaked features excluded from models to prevent proxy leakage.
- Two complementary analysis approaches are used:
    1. Statistical / Entropy-based tunneling detection using Shannon entropy analysis focused on DNS and HTTP protocol tunneling.
    2. Machine-learning-based attack analysis across 11 categories using algorithms like RandomForest and XGBoost, evaluated via 5-Fold Stratified Cross-Validation.
- The main evaluation metric is the Macro-F1 score (aiming >80%) for all classification tasks, completely replacing Accuracy to account for rare classes like data exfiltration.
- Furthermore, the scope includes an LLM Deception Layer: A fine-tuned language model (Phi-3-mini-4k-instruct via LoRA adapter) is integrated to process unknown SSH commands in Cowrie and generate context-aware synthetic Debian terminal outputs, extending session durations for better behavioral intelligence.
- At the end of the project, a local attack dataset, the analysis code and documentation will be shared as open source on GitHub, and the results will be used to produce academic publications.

The project is planned as an 8-month study, and the system is designed not as an active defense or prevention mechanism, but as a research and monitoring platform.

1.3 Definitions, Acronyms, and Abbreviations

- **Anomaly Detection** – Identifying patterns that deviate from normal system behavior.
- **Attack Classification** – Assigning detected events to categories such as brute force, port scanning, or bot activity.
- **Cowrie** – A medium-interaction SSH/Telnet honeypot capturing attacker behavior.
- **DMZ (Demilitarized Zone)** – A network segment that isolates publicly reachable devices from the internal network.
- **DNS/HTTP Tunneling** – Hiding data within DNS or HTTP traffic to bypass security controls.
- **Entropy (Shannon Entropy)** – A statistical measure of randomness used for tunneling detection.
- **Feature** – A numerical attribute extracted from logs (e.g., URI length, entropy score).
- **Honeypot** – A decoy system designed to attract and log malicious activities.

- **Honeytrap** – A low-interaction honeypot simulating common network services.
- **IoT (Internet of Things)** – A network of connected physical devices capable of communication.
- **Machine Learning (ML)** – Algorithms that learn from data to detect or classify attack behaviors.
- **MQTT** – A lightweight publish/subscribe protocol used to transfer logs.
- **SQLite** – A lightweight database storing raw logs and extracted features.
- **Suricata** – An IDS generating network security alerts and packet-level logs.
- **Data Leakage -** The unintentional use of heuristic or rule-based information in the ML training process that allows models to deterministically memorize labels instead of learning behavioral patterns.
- **Macro-F1 Score -** An evaluation metric that calculates the F1-score for each class independently and averages them equally, highly suited for imbalanced datasets (e.g., rare attacks like data exfiltration).
- **Offline Analysis / Batch Processing -** The process of aggregating stored raw logs into session-level intelligence (Parquet datasets) periodically or retrospectively, rather than evaluating network packets in real-time.
- **LLM (Large Language Model) -** In this project, an AI model (Phi-3) used to generate synthetic, dynamic responses to attacker commands to prevent honeypot detection.
- **SAFE Features vs. HEURISTIC Features -** SAFE features (e.g., payload entropy) are used for ML training, while HEURISTIC features (e.g., known command regex matches) are used for rule-based labeling and excluded from ML to prevent leakage.
- **Cross-Validation (5-Fold Stratified) -** A validation technique used to ensure model stability and generalizability by training and testing the model across 5 distinct data splits.

1.4 Document Overview

This SRS is structured as follows:

- **Section 2 — Overall Description:**
  Provides the system context, architecture perspective, user characteristics, constraints, and operating environment.
- **Section 3 — System Features:**
  Defines detailed functional requirements for each subsystem, including edge collection, offline ML preparation, data leakage prevention, and the newly integrated LLM-powered deception mechanisms.
- **Section 4 — External Interface Requirements:**
  Describes user interfaces, hardware interfaces, software interfaces, and communication interfaces.

- **Section 5 — Non-Functional Requirements:**
  Outlines performance goals, security requirements, usability, maintainability, and other critical constraints.
- **Section 6 — Other Requirements:**
  Includes ethical considerations, monitoring needs, and data-handling requirements.
- **Section 7 — Appendices:**
  Contains the glossary, sample logs, and feature definitions.

2. Overall Description

2.1 Product Perspective

The system is designed as a comprehensive IoT honeypot-based behavioral intelligence and deception platform. It operates in a real-world network setup by exposing a Raspberry Pi edge device through a DMZ and guest network, ensuring the capture of genuine attack traffic.

The architecture has evolved into an extended four-stage pipeline:

1. **Edge Honeypot Layer** (on Raspberry Pi):
   - Cowrie (SSH honeypot)
   - Honeytrap (HTTP/FTP emulation)
   - Suricata (IDS for network traffic inspection)
   - A local aggregator normalizes raw events and pushes them to a FIFO pipe.
2. **Data Transfer and Storage Layer:**
   - MQTT for lightweight log transmission
   - SQLite for storing all raw logs and extracted features
3. **Analysis and Visualization Layer:**
   - Constructs temporally coherent sessions from raw logs (e.g., 30-minute inactivity threshold).
   - Performs strict Data Leakage Prevention by separating 52 extracted features into SAFE and HEURISTIC categories.
   - Utilizes machine learning models (e.g., XGBoost, Random Forest) to classify 11 attack types, tunneling, and bot vs. manual behavior.
4. **LLM Deception Pipeline (Integration Layer):**
   - Employs a fine-tuned causal language model (Phi-3-mini-4k-instruct via LoRA adapter) to dynamically generate contextually appropriate Linux terminal outputs for unrecognized Cowrie commands, enhancing attacker engagement.

2.2 Product Functions

In general, the system performs the following functions:

- **Capture real attack traffic** through honeypots running different protocols.
- **Record attacker interactions** including commands, session metadata, and network packets.
- **Transfer log data** to the central system using MQTT.
- **Store logs** in a structured form in SQLite.
- **Extract meaningful traffic features**, such as entropy, URI length, session duration, and connection intensity.
- **Session Construction,** group discrete log events into continuous attacker sessions based on source/destination IP pairs and time gaps.
- **Leakage-Aware Feature Extraction:** Extract 52 behavioral and heuristic features, automatically filtering out features that cause direct or proxy data leakage before ML training.
- **Detect DNS/HTTP tunneling** using entropy-based statistical methods.
- **Classify cyber attacks** (brute force, port scan, malware download, bot/human).
- **Rigorous Evaluation:** Validate ML performance using 5-Fold Stratified Cross-Validation and Macro-F1 scores, addressing extreme class imbalances.
- **LLM-Powered Analytics:** The fine-tuned model was used to measure how long the attacker could remain on the system after being responded to using the LLM.

2.3 User Classes and Characteristics

The system is intended for the following user groups:

- **Security Researcher / SOC Analyst:**
  Uses the offline session intelligence datasets (Parquet formats) to examine attacker TTPs (Tactics, Techniques, and Procedures), study behavioral patterns, and evaluate ML classification outputs.
- **System Maintainer / ML Engineer:**
  Installs honeypots, manages the MQTT/SQLite infrastructure, maintains the LLM Gateway integration, and retrains models using the ML preparation pipelines.

Both user types are expected to have basic knowledge of network security, Linux environments, and data analysis.

2.4 Operating Environment

The system operates in the following hardware and software environments:

Hardware

- Raspberry Pi 4 Model B (4GB RAM)

- Home/office router providing DMZ and guest network structure
- Central computer/server used as the analysis/ local database storage
- Cloud/Local GPU environments (Google Colab T4) for fine-tuning the LLM

Network Setup

- Raspberry Pi exposed to the internet via Guest Network + DMZ
- Direct connection to the internet to capture external real-world attacks

Software

- Cowrie, Honeytrap, Suricata honeypot/IDS tools
- MQTT broker
- SQLite database
- Python (Pandas, Scikit-learn, XGBoost) for analysis, preprocessing, and ML
- HuggingFace ecosystem (QLoRA, Phi-3 adapter)

2.5 Design and Implementation Constraints

The system must operate under the following constraints:

- **Low-resource hardware:** Raspberry Pi has limited CPU, memory, and storage.
- **Real attack exposure:**
  The honeypot is directly exposed to the internet; configuration must avoid impacting any production network.
- **Log volume variability:**
  Real attack traffic may fluctuate; logs depend on attacker activity.
- **Strict Data Leakage Prevention:**
  The ML pipeline must absolutely exclude heuristic markers (e.g., regex matches) and direct score formulas (e.g., automation score) from the training features to ensure models learn genuine behavior.
- **Dependency on open-source tools:**
  The system relies entirely based on open-source software (Cowrie, Honeytrap, Suricata, MQTT, SQLite).
- **Strict evaluation targets:**
  DNS/HTTP tunneling detection: ≥ 80% Macro-F1 score
  Attack classification: ≥ 80%Macro-F1 score

2.6 Assumptions and Dependencies

The system is based on the following assumptions:

- The Raspberry Pi will successfully receive sufficient external attack traffic once exposed to DMZ/Guest network.
- MQTT communication between the Raspberry Pi and central computer remains stable.
- SQLite storage is sufficient for the volume of logs produced during the project.
- Statistical and machine learning analyses will be performed using Python on the central machine.
- The integration of the LLM pipeline assumes a reliable connection to the inference model and strict output filtering to prevent prompt injection or hallucinated system states.
- In case insufficient real-world data is collected, public datasets (CTU-13, CIC-IDS 2017) may be used as backup for training/testing, as stated in the risk management section.

3. System Features (Functional Requirements)

3.1 Edge Honeypot Operation

Description

The Raspberry Pi device runs honeypots (Cowrie, Honeytrap) and an IDS (Suricata) to capture actual attack traffic.

Inputs

- Incoming attack traffic from the internet via DMZ / guest network.
- Attacker commands, sessions, HTTP/FTP interactions, network packets.

Process / Behavior

- The honeypots interact with attackers and record their actions.
- Suricata inspects network packets and produces alerts/logs.

Outputs

- Log files containing timestamped attacker activities, commands, sessions, and alerts.

Functional Requirements

ID	Description
FR-1.1	The system shall run Cowrie to log SSH-based attacks and attacker commands.
FR-1.2	The system shall run Honeytrap to emulate HTTP/FTP services and collect associated attack traffic.
FR-1.3	The system shall run Suricata to generate IDS alerts and inspect network traffic.
FR-1.4	The system shall timestamp and store all attacker interactions in log files.
FR-1.5	The Raspberry Pi shall operate in DMZ/guest network mode to capture real external traffic.

3.2 Log Collection and Aggregation

Description

Log files produced by Cowrie, Honeytrap, and Suricata are collected and prepared for transmission.

Inputs

- Raw logs from the edge honeypots.

Process / Behavior

- All logs are gathered, combined if necessary, and queued for MQTT transmission.
- Normalized events are pushed into a shared thread-safe FIFO pipe.

Outputs

- Standardized log entries for MQTT transfer.

Functional Requirements

ID	Description
FR-2.1	The system shall collect logs from Cowrie, Honeytrap, and Suricata.
FR-2.2	The system shall merge or organize logs to ensure they can be transmitted through MQTT.
FR-2.3	Each log entry shall include the required metadata (timestamp, source, session data, etc.).

3.3 MQTT-Based Log Transmission

Description

The Raspberry Pi sends collected log data to the central analysis computer.

Inputs

- Logs from Section 3.2.

Process / Behavior

- Logs are published to MQTT topics.
- The central server subscribes and receives the logs.

Outputs

- Logs delivered to the central analysis machine.

Functional Requirements

ID	Description
FR-3.1	The system shall transmit logs to the central server using the MQTT protocol over TLS.
FR-3.2	The system shall ensure uninterrupted log forwarding while the honeypot is active.
FR-3.3	The system shall retry transmission in case of temporary connectivity issues.

3.4 Central Log Storage (SQLite)

Description

All incoming logs are stored in a structured SQLite database.

Inputs

- Log entries received via MQTT.

Process / Behavior

- Logs are parsed, validated, and inserted into SQLite tables.

Outputs

- Structured datasets ready for offline session building and feature extraction.

Functional Requirements

ID	Description
FR-4.1	The system shall store all received log data in a SQLite database.
FR-4.2	The system shall create tables corresponding to each log type or analysis requirement.
FR-4.3	The system shall validate each entry (e.g., formatting, required fields) before storage.

3.5 Offline Session Construction

Description

Instead of real-time packet inspection, the system processes logs offline to construct behavioral sessions representing complete attacker interactions.

Inputs

- Log entries stored in SQLite.

Process / Behavior

- Group events by (src_ip, dest_ip) pairs.
- Apply a 30-minute inactivity gap threshold to define session boundaries.

Outputs

- Temporally coherent sessions assigned with unique session IDs.

Functional Requirements

ID	Description
FR-5.1	The system shall perform offline aggregation of raw logs into behavioral sessions.
FR-5.2	The system shall define a new session if the inactivity gap between events exceeds 30 minutes.
FR-5.3	The system shall assign a deterministic session ID to each grouped sequence.

3.6 Feature Extraction and Rule-Based Labeling

Description

The system extracts a large set of measurable traffic characteristics from the sessions and applies deterministic rules to generate ground-truth labels.

Inputs

- Preprocessed data and constructed sessions.

Process / Behavior

- Compute 52 raw features across 5 categories (command, timing, network, credential, tunneling/payload).
- Calculate an automation_score to label sessions as Bot, Manual, or Mixed.
- Apply rule-based logic to assign 11 multi-label attack categories (e.g., tunneling, brute force, lateral movement).

Outputs

- Session datasets formatted as **.parquet** files containing features and labels.

Functional Requirements

ID	Description
FR-6.1	The system shall extract 52 raw features per session, including Shannon entropy for payloads and commands.
FR-6.2	The system shall assign 11 multi-label attack categories using deterministic heuristic rules.
FR-6.3	The system shall compute a continuous automation score to classify bot vs. manual behavior.
FR-6.4	The system shall export the intelligence data as **.parquet** and **.json** files.

3.7 Data Leakage Prevention

Description

To ensure the ML models learn genuine behavior rather than memorizing deterministic rules, the system explicitly filters out features that cause data leakage.

Inputs

- Extracted features and derived labels.

Process / Behavior

- Separate features into "SAFE" (behavioral) and "HEURISTIC" (rule-based/leaked) categories.

- Perform automated leakage checks to ensure features like **automation_score** or **credential_attempts** are excluded from the training set.

Outputs

- Leak-free feature sets (24 SAFE features) ready for ML training.

Functional Requirements

ID	Description
FR-7.1	The system shall separate extracted features into SAFE and HEURISTIC categories.
FR-7.2	The system shall perform automated leakage checks to exclude features that directly or indirectly determine a label from the ML training set.
FR-7.3	The system shall strictly use only the designated SAFE features (e.g., 24 features) to train the machine learning models.

3.8 Machine-Learning-Based Attack Classification

Description

ML models classify cyber attack behaviors using the leak-free offline datasets.

Inputs

- Leak-free SAFE feature dataset.

Process / Behavior

- Train models (e.g., XGBoost, Random Forest) for bot detection, tunneling detection, and 11-category multi-label attack classification.
- Apply Anomaly Detection (Isolation Forest) for extremely rare classes (e.g., data exfiltration).
- Evaluate models using 5-Fold Stratified Cross-Validation to ensure stability.

Outputs

- Trained models, classification results, and robust evaluation metrics (Macro-F1, ROC-AUC).

Functional Requirements

ID	Description
FR-8.1	The system shall train tree-based ML models (e.g., RandomForest, XGBoost) to classify sessions.
FR-8.2	The system shall classify bot vs. manual behavior, tunneling, and 11 specific attack types.
FR-8.3	The system shall employ anomaly detection techniques (e.g., Isolation Forest) for rare attack classes with insufficient positive samples.
FR-8.4	The system shall evaluate classifier performance primarily using the Macro-F1 score, targeting ≥ 90% for behavioral classifications.

FR-8.5	The system shall implement 5-Fold Stratified Cross-Validation to verify metric stability.

3.9 LLM-Powered Cowrie Deception (Integration Layer)

Description

To prevent attackers from easily fingerprinting the static Cowrie environment, an LLM generates dynamic, context-aware responses to unrecognized commands.

Inputs

- Unrecognized SSH commands entered by attackers into the Cowrie honeypot.

Process / Behavior

- The Command Router intercepts unknown commands and queries a fine-tuned LLM Gateway (Phi-3 via LoRA adapter).
- The LLM generates a synthetic Debian terminal output based on a predefined system prompt.
- An Output Filter sanitizes the response before returning it to the attacker.

Outputs

- Realistic, dynamic terminal outputs displayed to the attacker to extend session durations.

Functional Requirements

ID	Description
FR-9.1	The system shall intercept unrecognized commands in Cowrie and route them to an LLM Gateway.
FR-9.2	The system shall utilize a fine-tuned causal language model (e.g., Phi-3) to generate contextually accurate Linux terminal outputs.
FR-9.3	The system shall apply an output filter to sanitize LLM responses, ensuring no real system state or sensitive data is disclosed.
FR-9.4	The system shall log all LLM-generated interactions in the LLM_Log table for further behavioral analysis.

4. External Interface Requirements

4.1 User Interfaces

The system provides two distinct types of interfaces: an analytical interface for security researchers and a deceptive terminal interface for attackers.

Analytical Interface (For Researchers/Analysts)

- Instead of real-time dashboards, researchers interact with the system's outputs through offline analytical environments (e.g., Jupyter Notebooks).

- The system exports comprehensive session intelligence as .parquet and .json files.
- These datasets display:
    - 11 classified cyber attack types (e.g., lateral movement, malware dropper, data exfiltration).
    - Behavioral distinctions (bot vs. manual behavior) based on automation profiling.
    - Extracted features clearly separated into SAFE and HEURISTIC categories.
    - Detailed performance evaluation metrics including Macro-F1, PR-AUC, and ROC-AUC.

Deceptive Interface (For Attackers)

- Attackers interacting with the Cowrie honeypot are presented with a synthetic, LLM-generated Debian terminal interface for commands that fall outside Cowrie's default vocabulary.

ID	Description
UI-1	The system shall export session-level intelligence and ML classification outputs as structured .parquet and .json files for offline analysis.
UI-2	The system shall present a realistic, context-aware Linux terminal interface to attackers using the LLM Gateway for unrecognized commands.
UI-3	The analytical outputs shall clearly present model performance using Macro-F1, PR-AUC, and ROC-AUC metrics instead of simple accuracy.

4.2 Hardware Interfaces

Raspberry Pi 4 Model B

- Runs Cowrie, Honeytrap, and Suricata.
- Exposed to the Internet via DMZ and guest network.
- Sends logs to the central server via MQTT.

Home/Office Router

- Provides DMZ and Guest Network configuration.
- Connects Raspberry Pi directly to external networks to capture real attacks.

Central Analysis Machine & GPU Environment

- Receives MQTT log packets and runs SQLite database.
- Executes Python-based offline ML processing (Feature Extraction, Leakage Check, XGBoost/RandomForest training).

- Interfaces with cloud/local GPU environments (e.g., Google Colab T4) for fine-tuning the LLM (Phi-3) via QLoRA.

ID	Description
HWI-1	The system shall interface with a Raspberry Pi 4 as its primary edge device.
HWI-2	The system shall rely on a router capable of DMZ/Guest Network configuration.
HWI-3	The Raspberry Pi shall interface with the central analysis computer via standard network connectivity.
HWI-4	The system shall support interfacing with GPU hardware (e.g., Google Colab T4) for LLM fine-tuning and inference tasks.

4.3 Software Interfaces

Honeypot and IDS Tools

- **Cowrie**: SSH interaction logging
- **Honeytrap**: HTTP/FTP service emulation
- **Suricata**: Network packet inspection and alert generation

Data Processing & ML Tools

- **MQTT Broker**: Receives and forwards logs (e.g., HiveMQ Cloud)
- **SQLite**: Stores raw logs and extracted features
- **Python Stack:** Pandas, Scikit-learn, and XGBoost used for data preprocessing, leakage-aware feature extraction, 5-Fold Cross-Validation, and ML classification.

LLM & Deception Software

- **HuggingFace Ecosystem:** Uses QLoRA and PEFT for loading and fine-tuning the microsoft/Phi-3-mini-4k-instruct model.

ID	Description
SW-1	The system shall interface with Cowrie, Honeytrap, and Suricata for log generation.
SW-2	The system shall interface with an MQTT broker for data transfer.
SW-3	The system shall store logs and features using SQLite.
SW-4	The system shall interface with the HuggingFace ecosystem to deploy the fine-tuned Phi-3 LoRA adapter for deception generation.
SW-5	The offline analysis pipeline shall use the XGBoost and Scikit-learn libraries for behavioral classification tasks.

4.4 Communication Interfaces

MQTT Communication

- Used to transmit logs from Raspberry Pi to central computer.
- Lightweight, suitable for IoT devices.
- Handles small, frequent log messages.

Network Exposure

- Raspberry Pi is exposed to the Internet through DMZ/Guest Network.
- Allows real attack traffic to reach the honeypots.

Database Interaction

- Local SQLite queries are used during the offline dataset construction (extracting events into sessions).

ID	Description
COM-1	The system shall use the MQTT protocol for log transmission.
COM-2	The Raspberry Pi shall communicate with the central analysis server over a standard IP network.
COM-3	The system shall allow honeypots and IDS tools to receive external attacks via Internet-exposed DMZ/Guest Network connections.

5. Non-Functional Requirements

5.1 Performance Requirements

The system must meet strict performance expectations for offline behavioral classification and LLM inference.

Performance Requirements

ID	Description
PR-1	The machine-learning classifiers shall completely abandon "Accuracy" and instead target a Macro-F1 score of ≥ 90% for bot/human classification and tunneling detection due to high class imbalance.
PR-2	The multi-label attack classification models (e.g., XGBoost, RandomForest) shall achieve a Macro-F1 score of ≥ 90% across the 11 identified attack categories.
PR-3	The offline session construction pipeline shall efficiently process and aggregate millions of raw log events into structured sessions (e.g., 60,000+ sessions) within reasonable batch execution times.
PR-4	The system shall maintain stable SQLite database performance as the log corpus grows (e.g., handling 1.5+ million events / ~6.2 GB without corruption).
PR-5	The LLM (Phi-3) must generate and filter synthetic terminal responses with minimal latency.

5.2 Reliability Requirements

The system must reliably capture real attack traffic, secure data during transit, and maintain honeypot uptime.

ID	Description
RL-1	The honeypots (Cowrie, Honeytrap, Suricata) shall operate continuously during the data-collection period unless intentionally stopped.
RL-2	MQTT log transmission shall reliably deliver logs from the Raspberry Pi to the central server without data loss under normal conditions.
RL-3	The system shall ensure that all logs received through MQTT are stored in SQLite without corruption.
RL-4	In case real attack traffic is insufficient, the system shall allow integration of external public datasets (CTU-13, CIC-IDS 2017), as specified in the risk-management plan.

5.3 Security Requirements

The project involves real attack exposure; therefore, security constraints are derived from the project text.

ID	Description
SEC-1	The Raspberry Pi shall remain isolated from production networks by operating only through the DMZ and guest network.
SEC-2	No part of the honeypot system shall be connected to any internal or sensitive university network
SEC-3	All raw logs, SQLite databases, and extracted .parquet datasets shall remain securely accessible only within the offline analysis environment.
SEC-4	The system shall ensure that honeypots do not relay or forward attacks to other systems (decoy only).
SEC-5	The LLM Output Filter must sanitize generated responses to prevent attackers from manipulating the AI to disclose system prompts or generate toxic outputs.

5.4 Usability Requirements

These requirements ensure that researchers and analysts can easily interpret the complex, offline ML outputs and intelligence datasets.

ID	Description
US-1	The exported intelligence datasets (.parquet and .json) shall be clearly structured for seamless integration with offline analysis tools like Jupyter Notebooks and Pandas.
US-2	The offline feature extraction module shall explicitly separate and clearly label SAFE features from HEURISTIC features to guide analysts and prevent accidental data leakage during future experiments.
US-3	The ML evaluation pipeline shall automatically generate comprehensive, human-readable classification reports detailing Macro-F1, Precision, Recall, and ROC-AUC per class.

5.5 Maintainability Requirements

The system should be maintainable throughout the 8-month project timeline.

ID	Description
MT-1	Honeypot, MQTT, and database configurations shall be modular to allow updates or resets without affecting the entire system.
MT-2	Python-based preprocessing, data leakage checks, and machine-learning scripts shall be structurally modular to allow easy modification and automated re-testing.
MT-3	The system shall log internal execution errors (e.g., failed MQTT transmission, LLM generation timeouts) to support rapid debugging.
MT-4	The code, dataset, and documentation shall be published as open-source on GitHub at project completion.

5.6 Extensibility Requirements

According to our project's long-term vision:

ID	Description
EX-1	The system shall be designed so it can incorporate logs from additional honeypots or IDS tools seamlessly into the Preprocessed Log schema.
EX-2	The ML pipeline architecture shall allow for replacing or adding new algorithms (e.g., semi-supervised methods or few-shot learning) without altering the underlying data extraction flow.
EX-3	The feature extraction module shall allow for the addition of new behavioral features while preserving the automated Leakage Check logic.
EX-4	The LLM Command Router shall be model-agnostic, allowing the replacement of Phi-3 with newer or more efficient LLMs in the future.

5.7 Portability Requirements

The architecture must adapt to changing computational environments.

ID	Description
PT-1	The offline analysis environment (Python + SQLite + HuggingFace Ecosystem) shall be deployable on any compatible Linux-based central server or cloud instance (e.g., Google Colab).
PT-2	The codebase shall be platform-independent, allowing migration to different machines if needed.
PT-3	The Raspberry Pi honeypot configuration shall be reproducible on similar low-resource IoT hardware.

6. Other Requirements

6.1 Ethical and Privacy Considerations

The project collects real attack traffic by exposing a Raspberry Pi device to the Internet through a DMZ and guest network. Because the system does not interact with legitimate users or private data, but only records malicious activities initiated by external attackers, the ethical considerations are limited to ensuring safe decoy operation and responsible AI usage.

ID	Description
ETH-1	The system shall record only traffic directed at the honeypot device and shall not collect or process any personal, private, or unrelated user data.

ETH-2	The system shall ensure that collected attack logs are used strictly for research, analysis, and academic purposes.
ETH-3	The system shall treat all collected attack data as research material and shall not publish any sensitive information beyond what is allowed in open-source datasets (e.g., anonymized logs).
ETH-4	The system shall ensure safe handling of attack samples (e.g., malware download attempts) without allowing them to execute or spread.
ETH-5	The LLM Output Filter must sanitize synthetic responses to ensure the model does not hallucinate and disclose real proprietary information, PII (Personally Identifiable Information), or toxic content to the attacker.

6.2 Monitoring and Logging Requirements

Monitoring and logging are key components of the project, as the goal is to collect high-value real-world attack data.

ID	Description
MLR-1	The system shall continuously monitor SSH, HTTP/FTP, and network traffic through Cowrie, Honeytrap, and Suricata.
MLR-2	All logs produced by honeypots and IDS tools shall be transmitted via MQTT to the central analysis computer.
MLR-3	The system shall store all logs, extracted features, and detection/classification outputs.
MLR-4	The system shall log offline performance evaluation metrics for the ML models, prioritizing Macro-F1, PR-AUC, and ROC-AUC over simple accuracy.
MLR-5	The system shall log precise time-based information (timestamps, inter-command delays) to enable the offline aggregation of discrete events into temporally coherent behavioral sessions.

6.3 Data Handling and Output Requirements

The project generates several categories of data: raw logs, preprocessed event datasets, session-level intelligence, separated ML features (SAFE vs. HEURISTIC), classification outputs, and synthetic LLM generation logs.

Raw Log Handling

DH	Description
DH-1	The system shall store raw logs from Cowrie, Honeytrap, and Suricata in SQLite without modification.

Feature and Preprocessing Data

ID	Description
DH-2	Extracted features (entropy, URI length, inter-command delays, etc.) shall be explicitly categorized and stored as either SAFE (for ML training) or HEURISTIC (for rule-based labeling and exclusion from ML).

Analysis Output

ID	Description
DH-3	The system shall export final tunneling-detection results, multi-label attack classifications, and performance metrics as structured .parquet and .json files.
DH-4	The output data format shall be fully compatible with offline analytical environments, such as Python Pandas DataFrames and Jupyter Notebooks.

Dataset Publication

The project explicitly states that the dataset and code will be shared openly at the end.

ID	Description
DH-5	At project completion, the system shall provide exportable datasets and analysis code suitable for publication as open-source material on GitHub.
DH-6	Shared datasets shall include real attack logs and extracted features, provided that no sensitive or harmful information is released.

7. Appendices

7.1 Appendix A — Glossary

This glossary summarizes the key technical terms used throughout this SRS, updated for the offline behavioral intelligence and LLM integration phases:

- **Anomaly Detection:** Identifying patterns that deviate from expected system behavior, specifically used (e.g., Isolation Forest) for rare attack classes like data exfiltration where positive samples are insufficient (<50).
- **Attack Classification:** Categorizing detected events into 11 multi-label categories (e.g., lateral movement, malware dropper, credential spray) and binary behavioral classes (bot vs. human).
- **Cowrie:** An SSH/Telnet honeypot that logs attacker interactions.
- **Data Leakage:** The unintentional inclusion of heuristic features or rule-based proxies in ML training data, allowing models to trivially memorize labels. Prevented in this system by separating SAFE and HEURISTIC features.
- **DMZ (Demilitarized Zone):** A network segment exposing the Raspberry Pi device to the Internet while isolating it from internal networks.
- **DNS/HTTP Tunneling:** Hiding data within DNS or HTTP traffic to bypass network protections.
- **Entropy (Shannon Entropy):** A randomness metric used to detect protocol tunneling.
- **Feature:** A numerical attribute extracted from aggregated sessions. Divided strictly into SAFE (for ML) and HEURISTIC/EXCLUDED (for rule-based labeling).

- **HEURISTIC Features:** Features derived from deterministic rules (e.g., regex matches, has_download_execute_chain) or formulaic proxies (e.g., automation_score). Excluded from ML training to prevent data leakage.
- **Honeytrap:** A low-interaction honeypot simulating HTTP/FTP services.
- **IoT (Internet of Things):** A network of connected, resource-limited devices.
- **LLM (Large Language Model):** Specifically the Phi-3-mini-4k-instruct model, fine-tuned via QLoRA to generate dynamic, context-aware deceptive Linux responses for the Cowrie honeypot.
- **Machine Learning (ML):** Algorithms used to detect or classify attack behaviors.
- **Macro-F1 Score:** The primary evaluation metric used in this system to replace simple "Accuracy." It equally weighs precision and recall across all classes, solving the severe class imbalance problem in honeypot traffic.
- **MQTT:** A lightweight publish/subscribe protocol used for log transmission.
- **Offline Session Analysis:** The methodology of constructing temporally coherent behavioral sessions (e.g., using a 30-minute inactivity threshold) from raw logs for retrospective ML training, rather than inspecting packets in real-time.
- **SAFE Features:** Genuine behavioral metrics (e.g., payload_entropy, cmd_count, min_inter_cmd_delay) used strictly to train the ML models.
- **SQLite:** A lightweight database storing logs and extracted features.
- **Sliding Window:** A technique for evaluating entropy across sequential time-based segments.
- **Suricata:** A network IDS generating alerts and packet-level logs.

7.2 Appendix B — Feature List

The system extracts 52 measurable characteristics from the constructed sessions. To ensure scientific rigor and prevent data leakage, these features are strictly separated into SAFE and HEURISTIC/EXCLUDED categories:

Independent Variables: SAFE Behavioral Features (Used for ML Training)

There are 24 leak-free features used to train the XGBoost and Random Forest models:

Category	Example Features	Rationale
Command	cmd_count, cmd_entropy, cmd_avg_length, cmd_max_length, avg_cmd_per_event	Captures the volume, diversity, and complexity of attacker inputs without relying on specific keywords.
Timing	min_inter_cmd_delay, max_inter_cmd_delay, log_duration	Captures behavioral speed limits (automated bursts vs. human hesitation) without proxying bot thresholds.

Network	port_range_span, has_sensitive_ports, protocol_count, dns_query_count, dns_avg_query_length, http_unique_uris	Identifies scanning and multi-protocol lateral movement profiles.
Credential	unique_usernames, password_entropy_avg	Identifies credential stuffing patterns and dictionary word complexities.
Payload/Tunneling	payload_entropy_avg, payload_entropy_max, avg_payload_length, has_port_forwarding, has_encoded_payloads, tls_on_non_standard_port, dns_tunnel_indicator, entropy_ratio	Identifies encrypted, encoded, or obfuscated payloads typical of tunneling and malware droppers.

Independent Variables: HEURISTIC / EXCLUDED Features (Excluded from ML)

There are 28 features completely excluded from ML training to prevent direct or proxy data leakage:

- **Direct Leakage/Formulaic:** automation_score, timing_variance_ratio, events_per_minute, cmd_unique_ratio, burst_count, unique_dest_ports, unique_passwords, credential_retry_count.
- **Proxy Leakage:** credential_attempts, avg_inter_cmd_delay, std_inter_cmd_delay.
- **Rule-Based Indicators (Regex)**: has_download_execute_chain, has_reverse_shell, has_recon_commands, has_persistence, has_file_exfiltration, has_destructive, has_internal_ip_scan, has_default_creds, has_sql_injection, has_cmd_injection, has_path_traversal, etc.

Dependent Variables (Classification Targets)

The system predicts the following labels based only on the SAFE features:

- **Binary Classification**: bot_label (bot vs. manual), tunnel_label (tunneling vs. normal).
- **Multi-Label Attack Classification (11 Categories):** bruteforce, malware_dropper, reconnaissance, lateral_movement, credential_spray, port_scan, data_exfiltration, destructive, network_probe, service_interaction.

7.3 Appendix C — Project Schedule Overview

Project timeline:

Time Period	Major Activities
Oct–Dec 2025	Edge Honeypot installation (Cowrie, Honeytrap, Suricata), MQTT setup, SQLite schema creation, and raw log ingestion (Phase 1 & 2 Completed).
Jan–Feb 2026	Offline session construction, 52-feature extraction, leakage-aware SAFE/HEURISTIC separation, and initial ML model training (Phase 3 Completed).
Feb–Mar 2026	Advanced ML Validation: 5-Fold Stratified Cross-Validation, Macro-F1 evaluation, Feature Importance Analysis, and Anomaly Detection for rare classes (Phase 4 Completed).
Mar–Apr 2026	LLM Deception Pipeline: Synthetic dataset generation, Phi-3 fine-tuning via

	QLoRA (Colab T4), and Cowrie integration/routing logic (Phase 5 In Progress).
May 2026	Advanced LLM evaluation (Consistency & Hallucination rates), open-source dataset & code publication, and final thesis defense reporting.

7.4 Appendix D — System Architecture Diagram
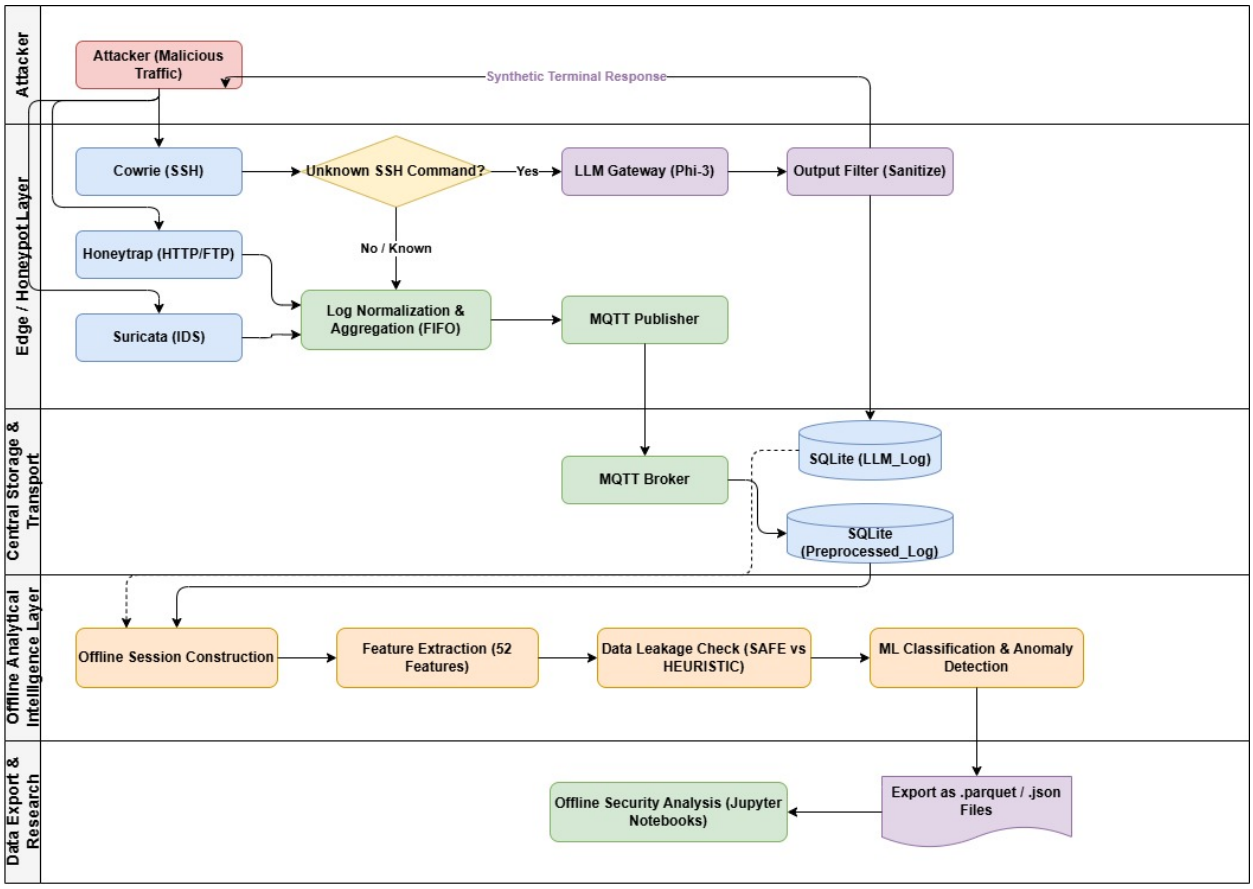
Figure F-1. System Data Flow Diagram